



Databases

Andrzej Nowicki

CERN School of Computing, Chile 2026

Lecture Outline

- Lecture I – past, present, future
- Lecture II – schema design, SQL
- Lecture III – scalability & reliability challenges**



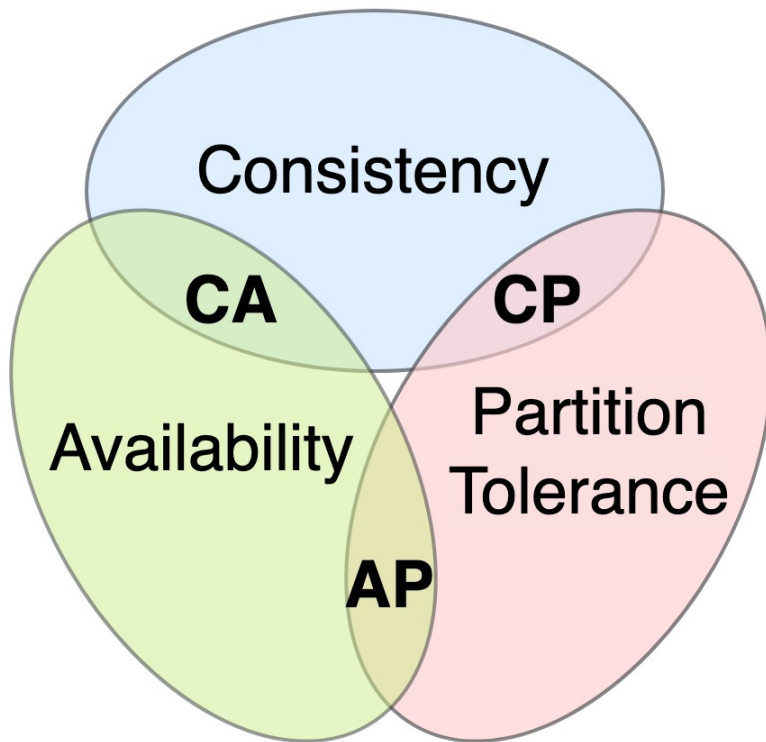
Scalability

There are two ways to achieve scalability:

- **Vertical scaling (scale-up)**
 - Adding resources like RAM, CPU, Disk Space to a single node
- **Horizontal scaling (scale-out)**
 - Adding more nodes
 - You can split your databases into smaller ones – **sharding**

CAP theorem AKA Brewer's theorem

You can only have 2 out of 3



Consistency

Every read receives the most recent write or an error. All clients see the same data at the same time, no matter which node they connect to.

Availability

Every request received by a non-failing node in the system must result in a response, without the guarantee that it contains the most recent version of the data.

Partition tolerance

The system continues to operate despite an arbitrary number of messages being dropped (or delayed) by the network between nodes.

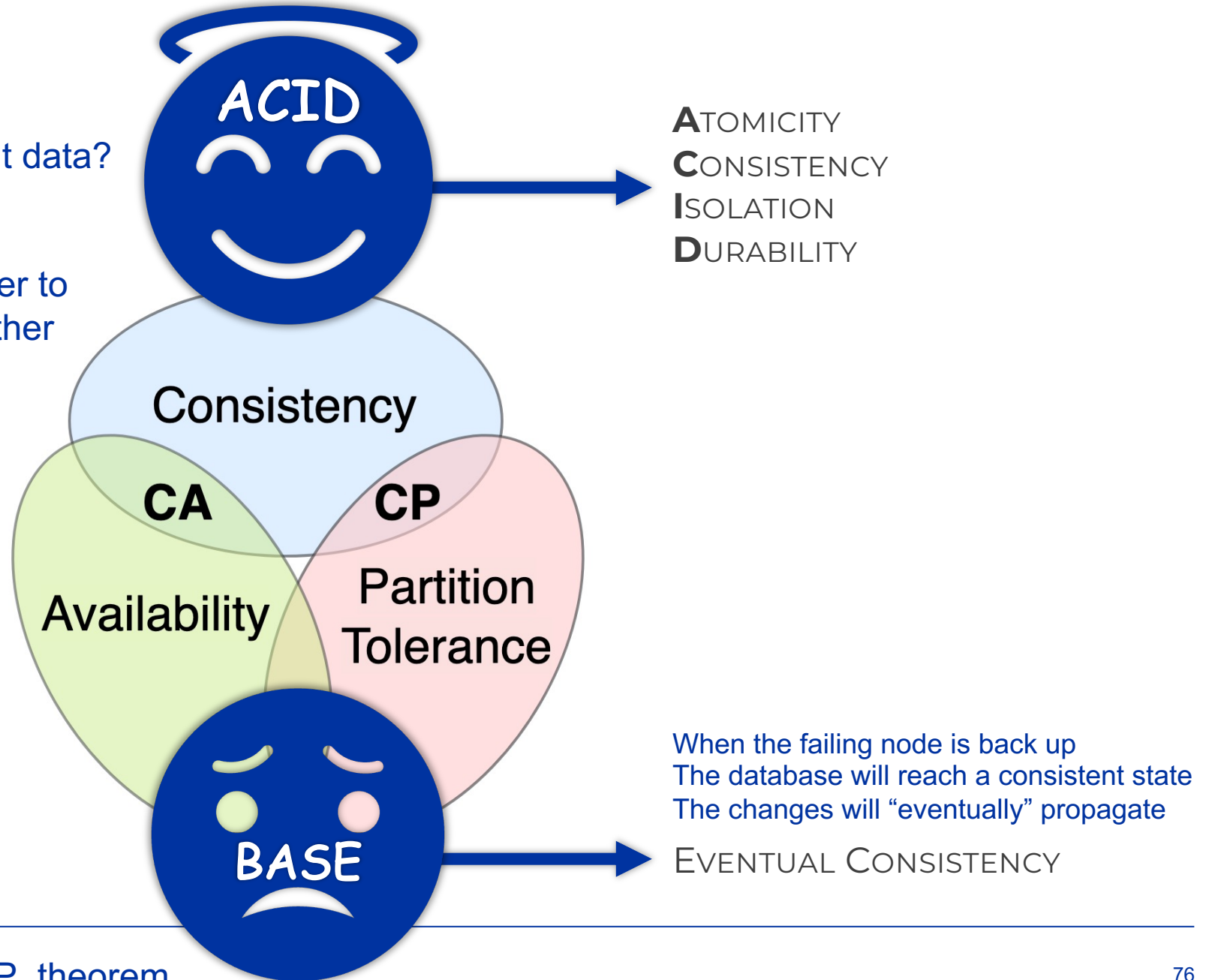
In case of network failure, it must be decided whether to do one of the following:

- cancel the operation and thus decrease the availability but ensure consistency
- proceed with the operation and thus provide availability but risk inconsistency.

CAP theorem

Why would you ever want inconsistent data?

Think about your instagram posts:
In case of failure of a DB, people prefer to see an inaccurate number of likes, rather than not seeing your post at all.



How to make your database resilient?

Basic database system

What if something breaks?



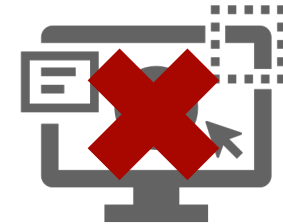
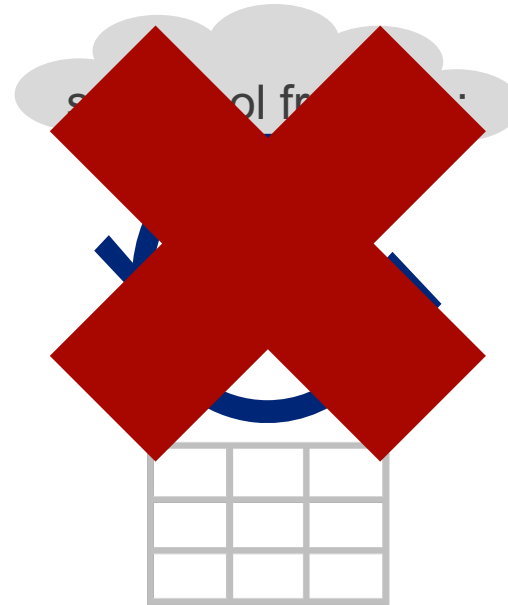
STORAGE



DATAFILES



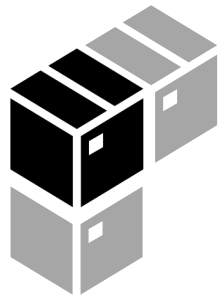
SERVER



APPLICATION

Highly available database system

Shared-Disk architecture e.g. Oracle Real Application Cluster

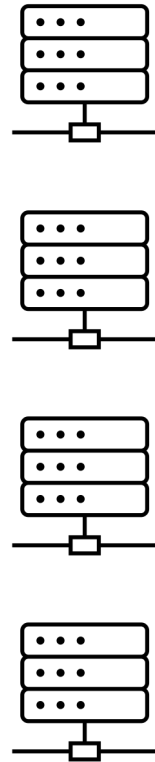


ENTERPRISE-LEVEL
STORAGE

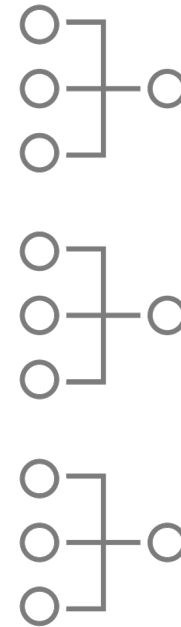
- SHARED ACROSS NODES
- SHOULDN'T HAVE SINGLE POINTS OF FAILURE



DATAFILES



SERVERS



LISTENERS
(LOAD BALANCERS)

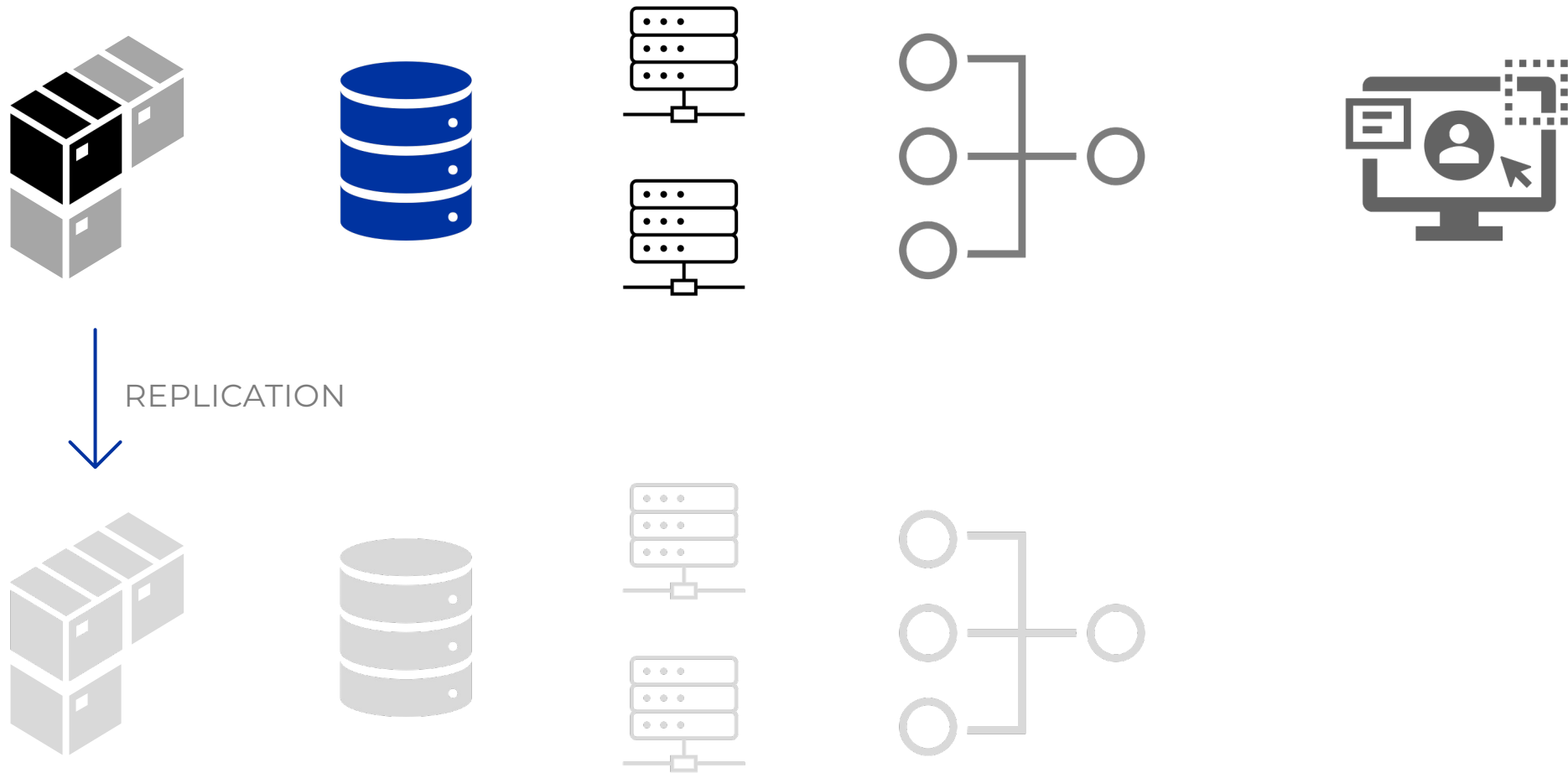


APPLICATION

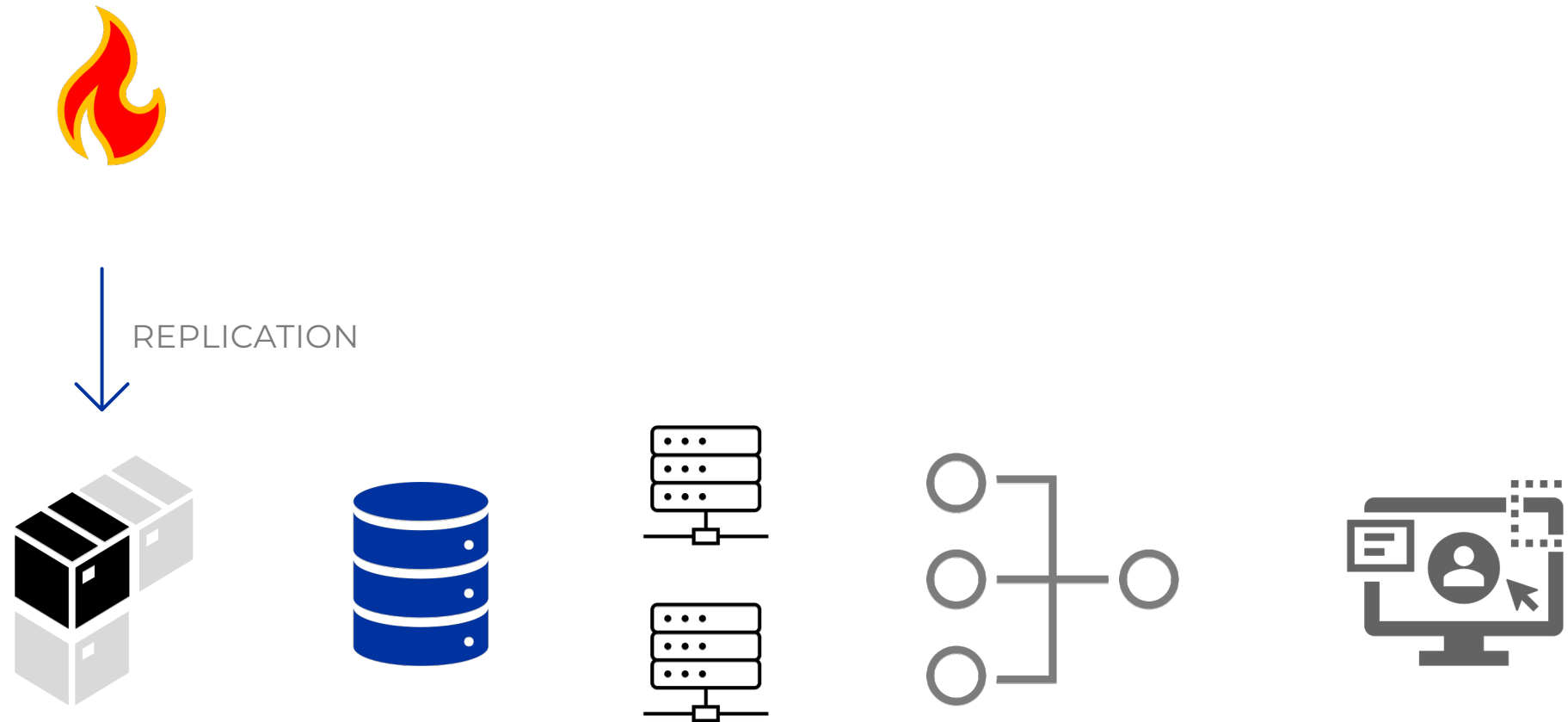
Is that enough?

What if your datacenter is on fire?

Highly available database system with DR

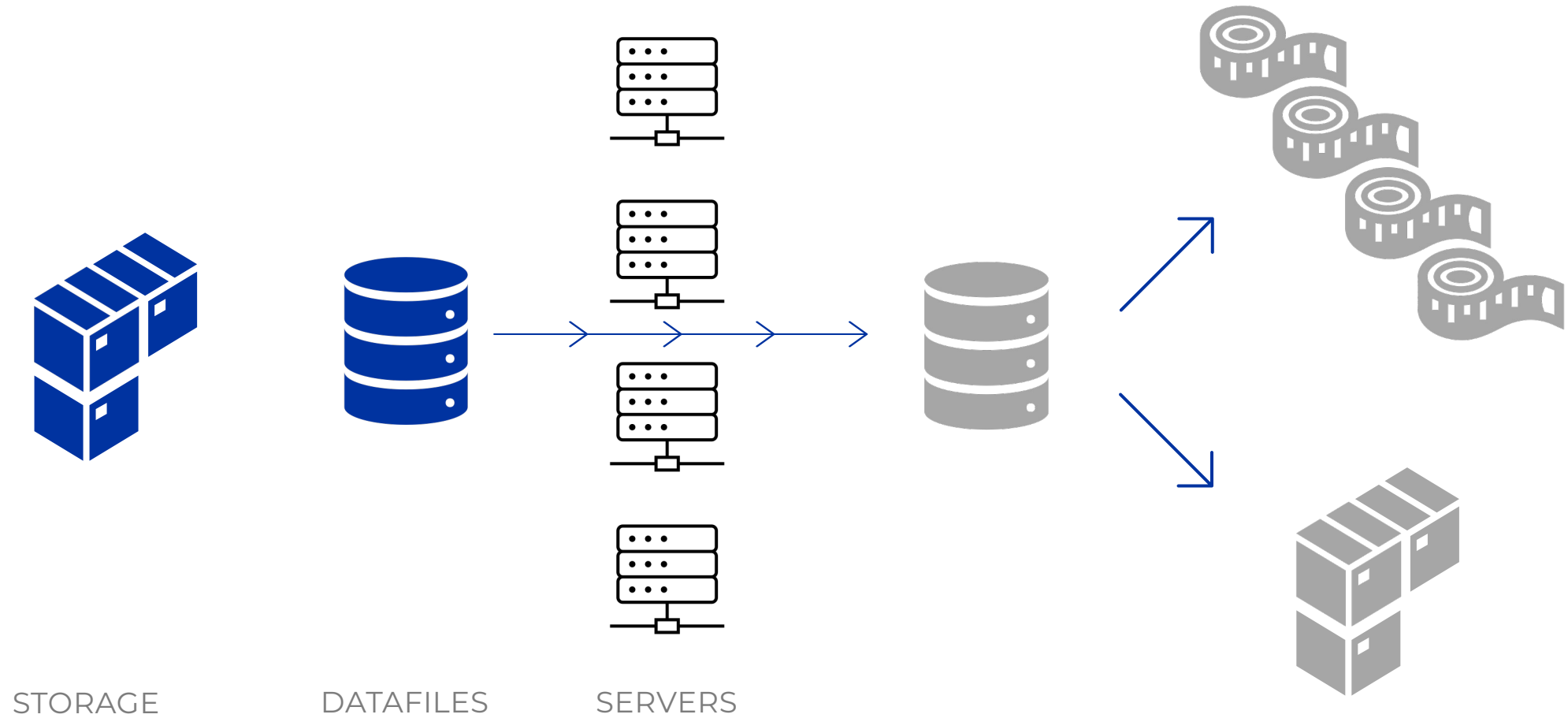


Highly available database system with DR



Is that enough?

Backups



Backups

In the past – backups were done in an offline fashion:

- 1. Stop your database (or disable writes)**
- 2. This ensures that you have a consistent state of your data on disk**
- 3. You make a 1:1 copy somewhere else**
- 4. Job done!**

Replication

How do you set up a replica?

Short explanation:

- 1. Take a backup**
- 2. Configure primary to “send all operations” to your replica**
- 3. Copy your backup to the replica**
- 4. Start your primary**
- 5. Configure the replica to “replay all operations”**
- 6. You have a 1:1 copy of your data which updates as your primary update**

Replication

The "stream of all operations" is called:

WAL (Write Ahead Log) in Postgres;

Redo Log in Oracle;

Binlog in MySQL.

You can configure your replication to be:

Asynchronous (user commits a transaction, the RDBMS replies COMMIT OK when it's written to disk; the transaction information is sent to the replica independently)

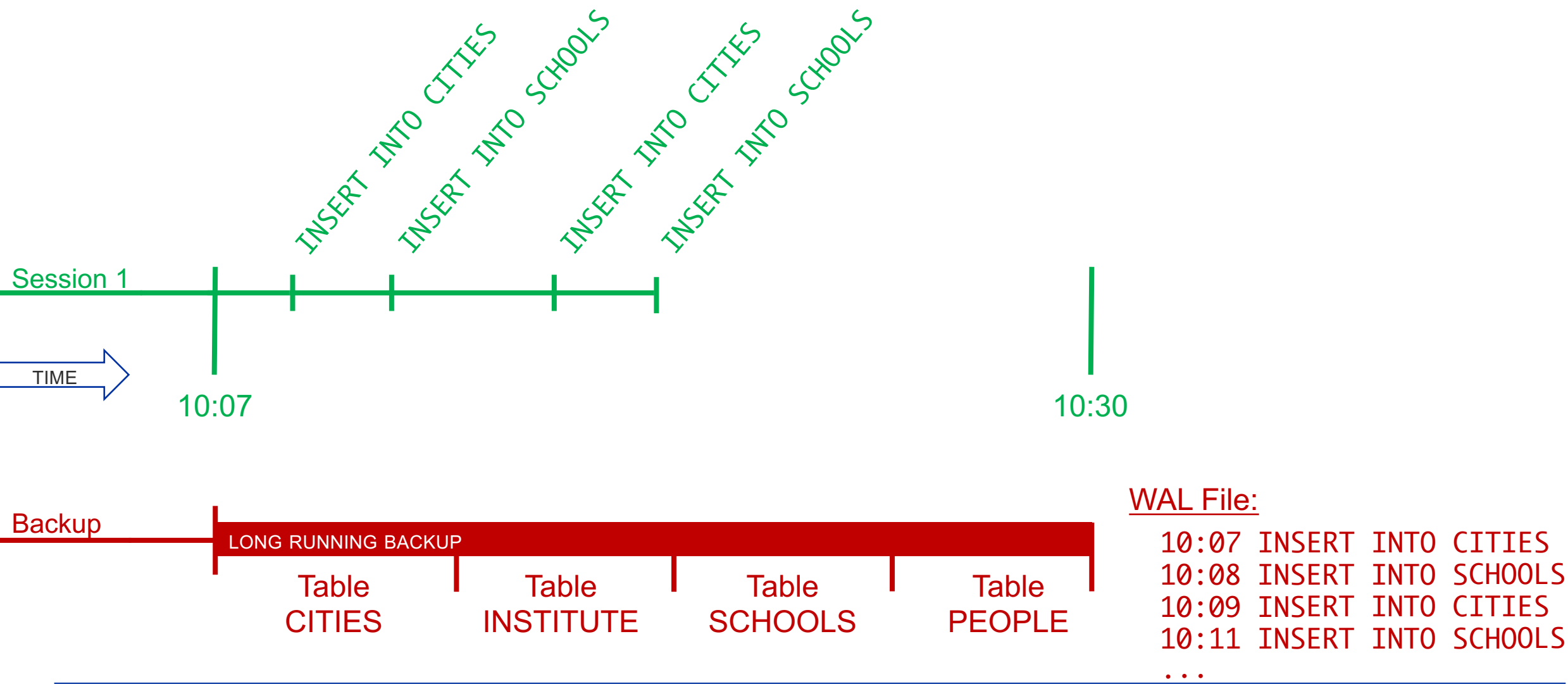
Synchronous (the RDBMS sends the operation to replica, waits for a confirmation from the replica before replying COMMIT OK to the application)

Online Backup

Modern database systems are allowing us to take backups without stopping the writes. That's what we call an online backup.

We need to backup the data files (which takes a long time, so the data will be inconsistent)... but also keep track of the log of all the changes which happened during that time.

Online Backup



Restore from online backup

Step 1. Restore tables from backup:

Data will be inconsistent, each table has state from a different moment in time



Step 2. Replay operations to get to a consistent state: (The recover operation)

You have a log of all of the changes done to a database

You rerun everything unless it was already applied

10:07 INSERT INTO CITIES
10:08 INSERT INTO SCHOOLS
10:09 INSERT INTO CITIES
10:11 INSERT INTO SCHOOLS
...
10:30 INSERT INTO SCHOOLS
10:31 INSERT INTO PEOPLE

Step 3. Database restored & recovered!

As a side effect...

If you backup ALL your “stream of operation” (WAL files)...



```
10:07 INSERT INTO CITIES
10:08 INSERT INTO SCHOOLS
10:09 INSERT INTO CITIES
10:11 INSERT INTO SCHOOLS
...
11:09 INSERT INTO CITIES
11:10 INSERT INTO CITIES


---


11:11 DROP TABLE SCHOOLS
...
```

You can do...

Point in Time Recovery

Step 1. Restore from backup older than the problematic time

Step 2. Recover (replay operations) to a point in time before your problem

Step 3. Profit!

Backups Summary

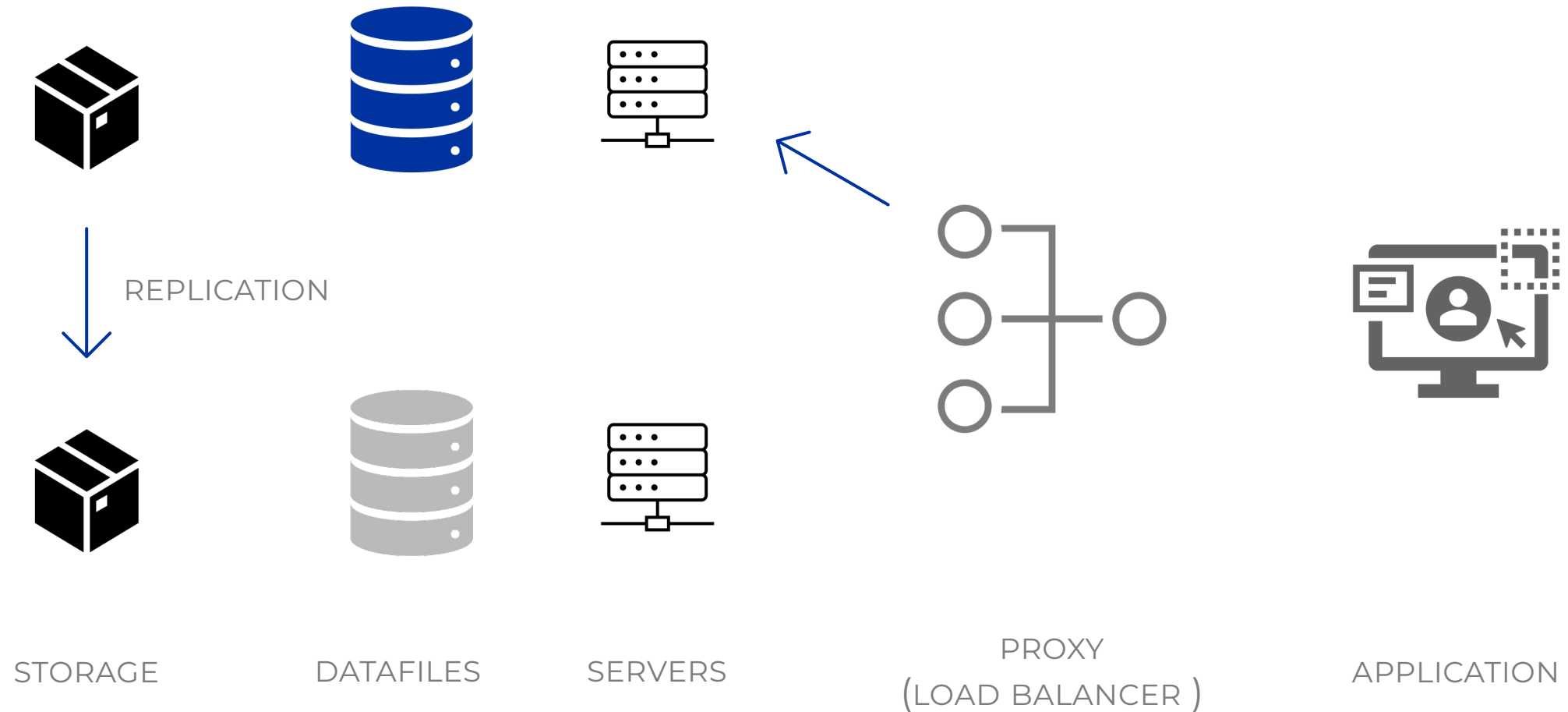
Always test your backup strategy!

A backup without a tested recovery strategy is worth nothing.

An online database backup without the WAL files is pretty much useless as the data will be inconsistent

Highly available database system

Shared-Nothing architecture e.g. Patroni for PostgreSQL



Partitioning

Splitting tables into smaller manageable pieces – partitions

Improves query performance by scanning only relevant partitions

Types of partitioning: Range (e.g. date ranges), List (e.g. countries), Hash (hash function)

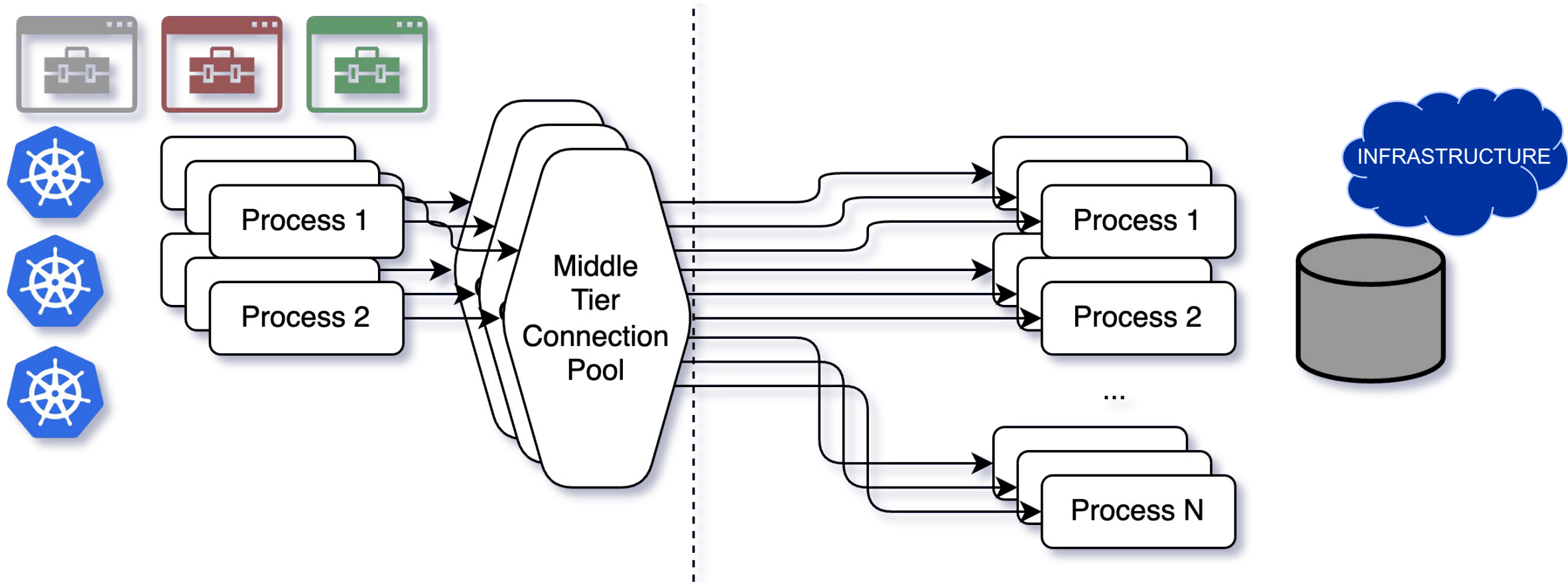
```
CREATE TABLE events (  
    id          serial PRIMARY KEY,  
    event_ts    timestamp NOT NULL,  
    data        jsonb  
) PARTITION BY RANGE (event_ts);
```

```
CREATE TABLE events_2026_01 PARTITION OF events  
    FOR VALUES FROM ('2026-01-01 00:00:00') TO ('2026-02-01 00:00:00');
```

```
CREATE TABLE events_2026_02 PARTITION OF events  
    FOR VALUES FROM ('2026-02-01 00:00:00') TO ('2026-03-01 00:00:00');
```

How does our infra look on the application side?

Application Side



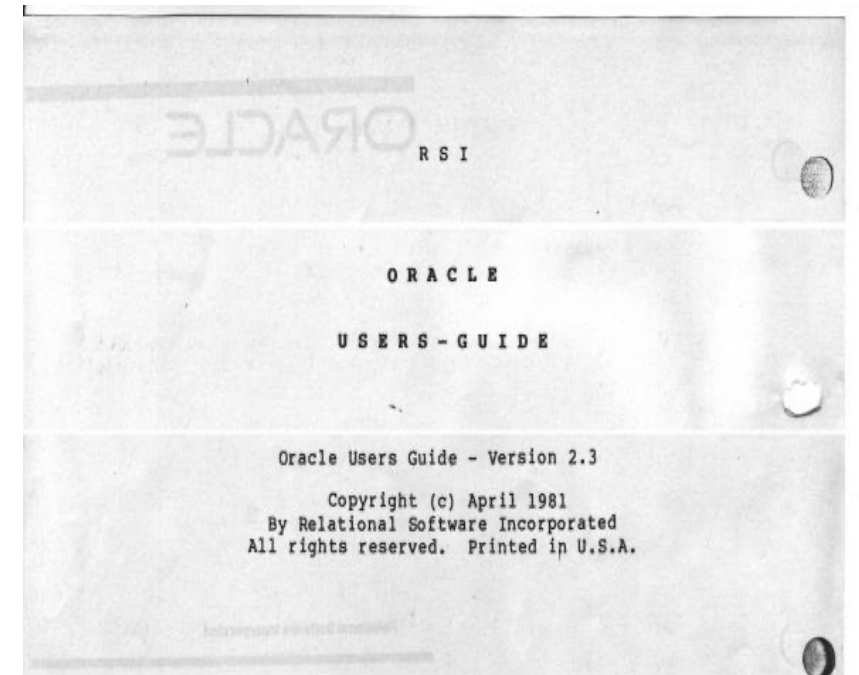
Databases at CERN

Oracle since 1982

- 105 Oracle databases, more than 12.000 Oracle accounts
- *RAC, Active Data Guard, GoldenGate, OEM, RMAN, APEX, Cloud...*
- Complex environment, ≈ 100 physical servers

Database on Demand (DBoD) since 2011

- ≈ 600 MySQL, ≈ 400 PostgreSQL, ≈ 200 InfluxDB
- *Automated backup and recovery services, monitoring, clones, replicas*
- *HA MySQL clusters (Proxy + primary replica)*
- *≈ 200 servers (both VMs and physical)*



Size of the database environment

		Total size
Oracle		≈5 PB
DBoD (MySQL, PostgreSQL, InfluxDB)		≈150 TB
Backups		≈3 PB

Which services at CERN rely on Oracle databases?

ChatGPT generated answer

Some key services at CERN that use Oracle databases include:

1. **Physics Data Management:** Particularly from the Large Hadron Collider (LHC), massive amounts of data are generated. Oracle databases help organize and facilitate access to large volumes of data for particle physics experiments.

Online and offline computing – e.g. ATONR, ATLR

2. **Accelerator Operations:** The LHC and other accelerators require precise control of operational parameters, monitored and adjusted in real-time. Oracle databases store historical analysis for optimizing accelerator performance.

ACCCON databases

3. **IT and Administrative Systems:** CERN uses Oracle databases for many internal administrative services, such as financial management, human resources, and procurement systems. These databases support the organization's daily administrative functions and ensure smooth, reliable operations.

e.g. EDH, LMS, INFOR, PAYROLL

4. **Data Archiving and Analysis:** Scientists and engineers access data for analysis over the long term. This service ensures that data from past experiments and operational activities remain accessible and organized for future use.

5. **Asset and Inventory Management:** All equipment, components, and materials are cataloged and tracked using Oracle databases.

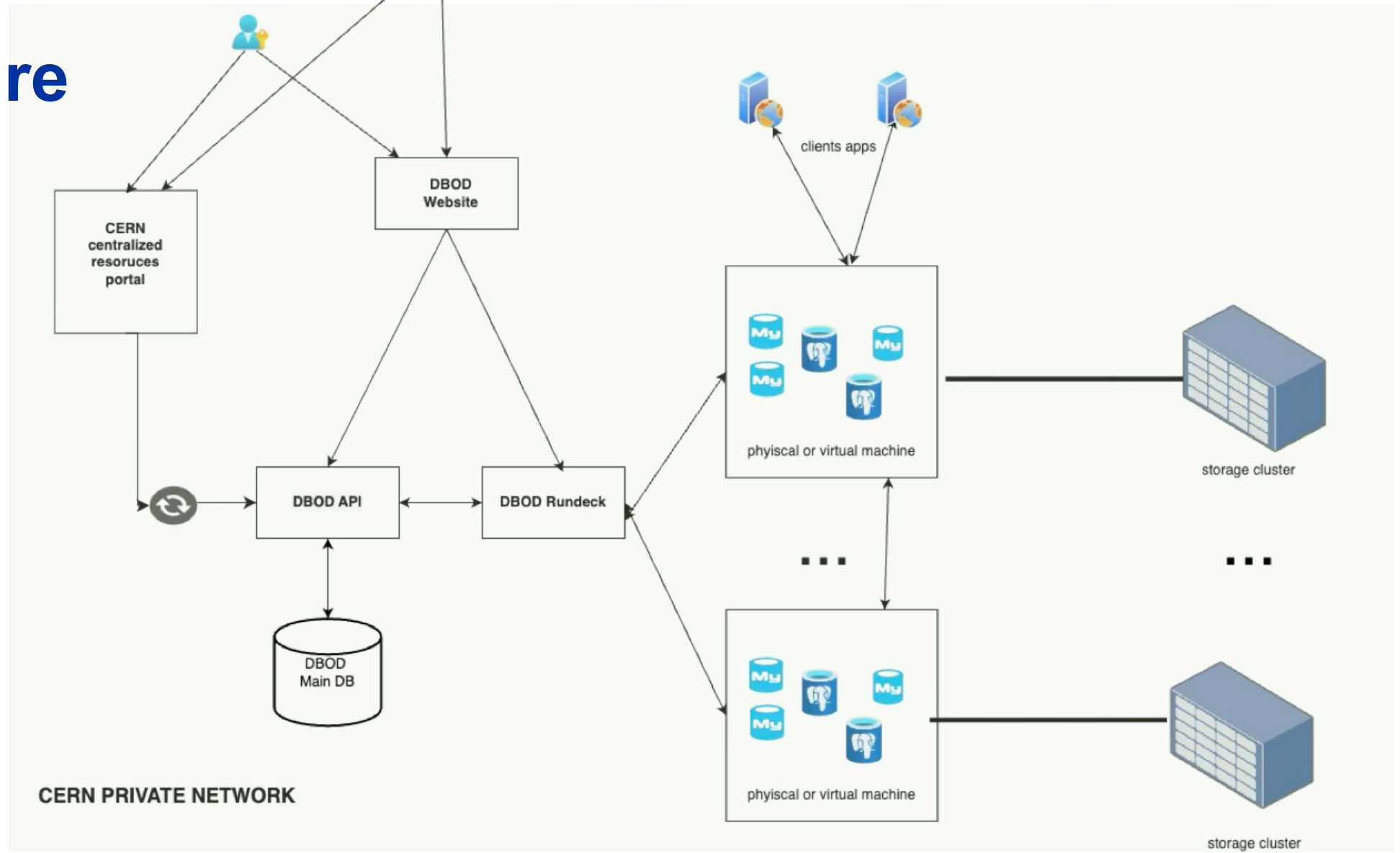
...many many more

6. **Project Management and Collaboration Tools:** CERN's project management platforms, often integrated with Oracle, help scientists and engineers plan, track, and collaborate on complex, large-scale projects across the organization.

Services relying on DBoD

- Indico (meeting planning, room booking system)
- Authorization Service (SSO, 2FA)
- Configuration Management (puppet, foreman), Secrets Vault
- Jira, Gitlab, Openstack
- Websites (<https://home.cern/>)
- CERN Document Server (<https://cds.cern.ch>)
- File Transfer System
- ATLAS Panda (Production and Distributed Analysis System)
- and more...

DBoD architecture



What do we do?

We automate things that are done often:

- Patching of database software
- Hardware migrations between servers
- New installations
- Accounts provisioning (and deletion)



Thank you !



andrzej.nowicki@cern.ch